

Informe final — Análisis del ejercicio

Fuente: documento_de_prompts_y_respuestas.md y constitución del proyecto.

1. ¿En qué incremento el agente de IA requirió más intervención humana y por qué?

Incremento 4 (Colaboración), junto con su integración con el Incremento 3. Fue donde más prompts de corrección hicieron falta:

- **Ramas paralelas.** Un compañero desarrollaba el Incremento 3 en otra rama, así que hubo que repetir la advertencia de respetar su trabajo. El agente no encontró el Incremento 3 en el repositorio y construyó el 4 desde `main`. Eso dejó pendiente una fusión manual: `flask db merge heads` y revisar `get_user_tasks` y `list.html`, que chocaban entre ramas.
- **Una desviación explícita.** Hubo que pedirle que revisara tres pruebas "en peligro". Eran advertencias de deprecación de SQLAlchemy (`Query.get()`), y las corrigió en una rama aparte.
- **Un segundo ciclo en la auditoría final.** Surgió el error `no such table: users` de una captura. `taskcontrol.db` estaba vacío y faltaba la migración del Incremento 3, y el código estaba duplicado por los merges (`Task` con definiciones dobles).

La causa no fue la complejidad de HU-10 y HU-11. El agente trabaja con el contexto de su rama y no ve lo que hace otra persona en paralelo. La coordinación entre ramas, el orden de fusión y las migraciones encadenadas siguieron dependiendo de una persona. Los Incrementos 1, 2 y 5, con contexto lineal, necesitaron casi una sola instrucción cada uno.

2. ¿Qué principio de la constitución fue más difícil de hacer cumplir en la práctica?

VII. Seguridad por defecto (validación y autorización en backend). Los incrementos pasaron todas sus pruebas, y aun así la auditoría final encontró incumplimientos:

- redirección abierta en el login;
- errores 500 al editar la tarea de otro usuario;
- fechas y categorías inválidas sin controlar;
- títulos sin límite de longitud;
- códigos HTTP incorrectos (400 en lugar de 404).

Nueve de las 73 pruebas de auditoría fallaban sobre el código original. Las pruebas por incremento cubrían el camino feliz, y el agente no aplicaba "validar exhaustivamente" por iniciativa propia. Se corrigió después, no se previno.

Otros dos principios también presentaron problemas:

- **VI (Migraciones):** se rompió en la integración de ramas. Faltó una migración y la base estaba vacía.

- **IV (Test-First):** el documento no muestra la fase Red, es decir, pruebas que fallan antes de implementar. Solo muestra los resultados finales, así que su cumplimiento real es difícil de comprobar.

3. ¿Qué harían distinto con otra herramienta o con otro framework?

- **Coordinación de ramas.** Definir el orden de fusión y la numeración de migraciones desde el inicio. Cada incremento partiría de una rama con el anterior ya integrado. Se añadiría integración continua que ejecute `pytest` y `flask db upgrade` desde una base vacía en cada PR, para que los conflictos aparezcan al integrar y no al final.
- **Principios verificables.** Convertir los principios de la constitución en checks automáticos: un linter para que las rutas no usen SQLAlchemy directamente, una prueba que valide las migraciones y una lista fija de casos negativos de seguridad por endpoint. Es más fiable que confiar en que el agente los recuerde.
- **Test-First demostrable.** Pedir que el agente guarde evidencia de la fase Red, por ejemplo en commits separados de pruebas y de implementación.
- **Auditoría más temprana.** Hacer una auditoría ligera al cierre de cada incremento, no solo una al final, para que los errores no se acumulen.
- **Otra herramienta o framework.** Se mantendría Flask por las restricciones de la constitución. Se probaría un framework con validación declarativa, como Pydantic o Marshmallow, que cubre parte del principio VII sin depender de que el agente escriba cada validación a mano. También se usaría un agente con memoria compartida, o varios agentes por rama, para reducir el desconocimiento del trabajo paralelo.

Nota sobre la conclusión

El cierre del documento de prompts considera el proyecto un éxito porque las pruebas pasaron. La auditoría final contradice en parte esa conclusión: encontró defectos reales que las pruebas por incremento no detectaban. El éxito fue posible gracias a esa auditoría, no a las pruebas iniciales.